

EKS Best Practice: Public-Facing Load Balancer for ingress-nginx

Cluster: "test-cluster" · Region: us-east-2 · Ingress controller: NGINX, listening on 80/443

Recommended architecture

Internet → Network Load Balancer (L4, internet-facing, IP target type) → ingress-nginx-controller pods → backend Services (via Ingress rules)

Use a Network Load Balancer (NLB), not an Application Load Balancer (ALB), in front of ingress-nginx. Nginx already performs Layer 7 routing (host/path rules) and TLS termination; putting an ALB in front would add a second, redundant L7 hop. An NLB operates at Layer 4, passes traffic straight through, preserves the client's source IP without needing proxy-protocol workarounds, and is the pattern recommended by both the AWS EKS Best Practices Guide and AWS's own engineering blog for exactly this "external LB + self-managed ingress controller" topology. Reserve ALB for when the AWS Load Balancer Controller itself is acting as your Ingress (i.e., no separate nginx layer).

Also move off `hostNetwork: true` / `hostPort 80/443` if that's how ingress-nginx is currently exposed. That's very likely the cause of the `FailedScheduling ... didn't have free ports` error seen earlier in this cluster — `hostNetwork` limits you to one ingress-nginx pod per node on that port. Switching to a normal Service fronted by an NLB with **IP target type** lets the AWS Load Balancer Controller register pod IPs (via the VPC CNI) directly as NLB targets, so pods schedule normally with no per-node port contention and can scale freely.

Step 1 · Install the AWS Load Balancer Controller

This controller watches Service/Ingress objects and provisions the actual AWS NLB/ALB resources. It runs as a Deployment in the cluster and needs an IAM role via IRSA (IAM Roles for Service Accounts).

1a. Get the cluster's OIDC issuer and associate an IAM OIDC provider

```
CLUSTER_NAME=test-cluster
REGION=us-east-2
ACCOUNT_ID=186579592072

OIDC_ISSUER=$(aws eks describe-cluster \
  --name $CLUSTER_NAME --region $REGION \
  --query "cluster.identity.oidc.issuer" --output text)
echo $OIDC_ISSUER
# e.g. https://oidc.eks.us-east-2.amazonaws.com/id/EXAMPLED539D4633E53DE1B71EXAMPLE

OIDC_ID=$(echo $OIDC_ISSUER | sed 's|https://||')

# Check whether an OIDC provider already exists for this cluster
aws iam list-open-id-connect-providers | grep $OIDC_ID || \
aws iam create-open-id-connect-provider \
  --url $OIDC_ISSUER \
  --client-id-list sts.amazonaws.com \
  --thumbprint-list 9e99a48a9960b14926bb7f3b02e22da2b0ab7280
```

(The thumbprint above is AWS's well-known root CA thumbprint used for EKS OIDC providers; `eksctl utils associate-iam-oidc-provider` does the same thing in one command if you have `eksctl` available.)

1b. Create the IAM policy for the controller

```
curl -o iam-policy.json \
  https://raw.githubusercontent.com/kubernetes-sigs/aws-load-balancer-controller/main/docs/install/iam_policy.json

aws iam create-policy \
  --policy-name AWSLoadBalancerControllerIAMPolicy \
  --policy-document file://iam-policy.json
```

1c. Create the IAM role trusted by the controller's Kubernetes service account

```
cat > trust-policy.json << EOF
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Principal": {
        "Federated": "arn:aws:iam::${ACCOUNT_ID}:oidc-provider/${OIDC_ID}"
      },
      "Action": "sts:AssumeRoleWithWebIdentity",
```

```

    "Condition": {
      "StringEquals": {
        "${OIDC_ID}:sub": "system:serviceaccount:kube-system:aws-load-balancer-controller",
        "${OIDC_ID}:aud": "sts.amazonaws.com"
      }
    }
  }
}
EOF

```

```

aws iam create-role \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --assume-role-policy-document file://trust-policy.json

aws iam attach-role-policy \
  --role-name AmazonEKSLoadBalancerControllerRole \
  --policy-arn arn:aws:iam::${ACCOUNT_ID}:policy/AWSLoadBalancerControllerIAMPolicy

```

1d. Create the Kubernetes service account, annotated with the IAM role

```

kubectl create serviceaccount aws-load-balancer-controller -n kube-system

kubectl annotate serviceaccount aws-load-balancer-controller -n kube-system \
  eks.amazonaws.com/role-arn=arn:aws:iam::${ACCOUNT_ID}:role/AmazonEKSLoadBalancerControllerRole

```

1e. Install the controller via Helm

```

helm repo add eks https://aws.github.io/eks-charts
helm repo update

helm install aws-load-balancer-controller eks/aws-load-balancer-controller \
  -n kube-system \
  --set clusterName=$CLUSTER_NAME \
  --set serviceAccount.create=false \
  --set serviceAccount.name=aws-load-balancer-controller \
  --set region=$REGION \
  --set vpcId=$(aws eks describe-cluster --name $CLUSTER_NAME --region $REGION --query "cluster.resourcesVpcConfig.vpcId" --output text)

kubectl get deployment -n kube-system aws-load-balancer-controller

```

Step 2 • Tag subnets for auto-discovery

The controller auto-discovers which subnets to place the NLB in based on tags. Tag your public subnets (and private ones too, if you also plan internal-facing load balancers):

```

# Public subnets used for the internet-facing NLB
aws ec2 create-tags --resources subnet-aaaa1111 subnet-bbbb2222 \
  --region $REGION \
  --tags Key=kubernetes.io/role/elb,Value=1 \
    Key=kubernetes.io/cluster/${CLUSTER_NAME},Value=shared

# Private subnets (only needed if you also provision internal load balancers)
aws ec2 create-tags --resources subnet-cccc3333 subnet-dddd4444 \
  --region $REGION \
  --tags Key=kubernetes.io/role/internal-elb,Value=1 \
    Key=kubernetes.io/cluster/${CLUSTER_NAME},Value=shared

```

Replace the subnet IDs with your actual public subnet IDs — find them with `aws ec2 describe-subnets --filters "Name=vpc-id,Values=<VPC_ID>"`.

Step 3 • Expose ingress-nginx via a Service annotated for NLB

Update (or patch) the `ingress-nginx-controller` Service in the `ingress-nginx` namespace to be `type: LoadBalancer` with these annotations:

```

apiVersion: v1
kind: Service
metadata:
  name: ingress-nginx-controller
  namespace: ingress-nginx
  annotations:

```

```
service.beta.kubernetes.io/aws-load-balancer-type: external
service.beta.kubernetes.io/aws-load-balancer-nlb-target-type: ip
service.beta.kubernetes.io/aws-load-balancer-scheme: internet-facing
service.beta.kubernetes.io/aws-load-balancer-cross-zone-load-balancing-enabled: "true"
service.beta.kubernetes.io/aws-load-balancer-healthcheck-protocol: HTTP
service.beta.kubernetes.io/aws-load-balancer-healthcheck-port: "10254"
service.beta.kubernetes.io/aws-load-balancer-healthcheck-path: /healthz

spec:
  type: LoadBalancer
  ports:
    - name: http
      port: 80
      targetPort: http
    - name: https
      port: 443
      targetPort: https
  selector:
    app.kubernetes.io/name: ingress-nginx
    app.kubernetes.io/component: controller
```

```
kubectl apply -f ingress-nginx-service.yaml
```

Annotation	Why it matters
aws-load-balancer-type: external	Tells Kubernetes to use the AWS Load Balancer Controller instead of the legacy in-tree (Classic ELB) provisioner.
nlb-target-type: ip	Registers pod IPs directly as NLB targets via the VPC CNI — avoids hostNetwork/hostPort entirely and removes the one-pod-per-node scheduling constraint.
scheme: internet-facing	Provisions a public NLB in your tagged public subnets. Use internal instead if this shouldn't be reachable from outside the VPC.
cross-zone-load-balancing-enabled	Spreads traffic evenly across pods regardless of which AZ they're in, rather than only balancing within each AZ.
healthcheck-port: "10254" , healthcheck-path: /healthz	Nginx exposes its built-in health endpoint on port 10254 — point the NLB's health checks at it instead of the traffic ports.

Step 4 • TLS termination

Two supported options — pick one:

- **Terminate TLS at nginx (recommended default).** Keep certificates and SNI-based routing entirely inside the cluster (e.g. via cert-manager + Let's Encrypt, or your own certs as Kubernetes Secrets referenced by each Ingress). The NLB passes port 443 straight through as raw TCP. This gives you the most flexibility for per-host certs and keeps the AWS side simple.
- **Terminate TLS at the NLB.** Modern NLBs support TLS listeners backed by an ACM certificate. Add `service.beta.kubernetes.io/aws-load-balancer-ssl-cert: <ACM_CERT_ARN>` and `service.beta.kubernetes.io/aws-load-balancer-ssl-ports: "443"` to the Service annotations. Simpler certificate management, but loses per-host SNI flexibility inside the cluster and means nginx receives decrypted HTTP.

Step 5 • Verify and retrieve the NLB

```
kubectl get svc -n ingress-nginx ingress-nginx-controller -o wide
# EXTERNAL-IP column shows the NLB's DNS name once provisioned (can take 2-3 minutes)

aws elbv2 describe-load-balancers --region $REGION \
  --query "LoadBalancers[?contains(DNSName, 'ingress-nginx')] | Type=='network']" \
  --output table

aws elbv2 describe-target-health --region $REGION \
  --target-group-arn $(aws elbv2 describe-target-groups --region $REGION \
    --query "TargetGroups[0].TargetGroupArn" --output text)
```

Step 6 • Point DNS at the NLB

```
NLB_DNS=$(kubectl get svc -n ingress-nginx ingress-nginx-controller \
  -o jsonpath='{.status.loadBalancer.ingress[0].hostname}')

NLB_ZONE_ID=$(aws elbv2 describe-load-balancers --region $REGION \
  --query "LoadBalancers[?DNSName=='${NLB_DNS}'].CanonicalHostedZoneId" --output text)

aws route53 change-resource-record-sets \
  --hosted-zone-id \
```

```
--change-batch '{
  "Changes": [{
    "Action": "UPSERT",
    "ResourceRecordSet": {
      "Name": "app.example.com",
      "Type": "A",
      "AliasTarget": {
        "HostedZoneId": "'${NLB_ZONE_ID}'",
        "DNSName": "'${NLB_DNS}'",
        "EvaluateTargetHealth": true
      }
    }
  }]
}'
```

Best-practice checklist

- **IP target type + VPC CNI** instead of `hostNetwork/hostPort` — removes the one-pod-per-node port constraint and lets ingress-nginx scale normally.
- **NLB, not ALB, in front of a self-managed ingress controller** — avoids a redundant L7 hop since nginx already does host/path routing.
- **Cross-zone load balancing enabled** so traffic is distributed evenly across all nginx replicas, not just those in the AZ that received the connection.
- **Health checks point at nginx's `/healthz` on port 10254**, not the traffic-serving ports, so a single failing backend doesn't get confused with an unhealthy nginx pod.
- **Correct subnet tagging** (`kubernetes.io/role/elb` + `kubernetes.io/cluster/<name>`) or explicit subnet IDs via annotation — otherwise the controller can't auto-discover where to place the NLB.
- **AWS WAF only attaches to ALB/CloudFront, not NLB**. If WAF is a requirement, put CloudFront in front of the NLB, or reconsider using ALB as the ingress instead of nginx.
- **Scope the IAM policy for the controller** to what `iam_policy.json` defines — don't broaden it; the controller only needs `ELBv2/EC2/ACM/WAF` describe-and-manage permissions, nothing account-wide.